

ing key questions and generating action sequences to ensure successful task completion.

- *Multi-path Reasoning.* In this strategy, the reasoning steps for generating the final plans are organized into a tree-like structure. Each intermediate step may have multiple subsequent steps. This approach is analogous to human thinking, as individuals may have multiple choices at each reasoning step. In specific, Self-consistent CoT (CoT-SC) [51] believes that each complex problem has multiple ways of thinking to deduce the final answer. Thus, it starts by employing CoT to generate various reasoning paths and corresponding answers. Subsequently, the answer with the highest frequency is chosen as the final output. Tree of Thoughts (ToT) [52] is designed to generate plans using a tree-like reasoning structure. In this approach, each node in the tree represents a "thought," which corresponds to an intermediate reasoning step. The selection of these intermediate steps is based on the evaluation of LLMs. The final plan is generated using either the breadth-first search (BFS) or depth-first search (DFS) strategy. Comparing with CoT-SC, which generates all the planned steps together, ToT needs to query LLMs for each reasoning step. In RecMind [53], the authors designed a self-inspiring mechanism, where the discarded historical information in the planning process is also leveraged to derive new reasoning steps. In GoT [54], the authors expand the tree-like reasoning structure in ToT to graph structures, resulting in more powerful prompting strategies. In AoT [55], the authors design a novel method to enhance the reasoning processes of LLMs by incorporating algorithmic examples into the prompts. Remarkably, this method only needs to query LLMs for only one or a few times. In [44], the LLMs are leveraged as zero-shot planners. At each planning step, they first generate

multiple possible next steps, and then determine the final one based on their distances to admissible actions. [56] further improves [44] by incorporating examples that are similar to the queries in the prompts. RAP [57] builds a world model to simulate the potential benefits of different plans based on Monte Carlo Tree Search (MCTS), and then, the final plan is generated by aggregating multiple MCTS iterations. To enhance comprehension, we provide an illustration comparing the strategies of single-path and multi-path reasoning in Figure 3.

- *External Planner.* Despite the demonstrated power of LLMs in zero-shot planning, effectively generating plans for domain-specific problems remains highly challenging. To address this challenge, researchers turn to external planners. These tools are well-developed and employ efficient search algorithms to rapidly identify correct, or even optimal, plans. In specific, LLM+P [58] first transforms the task descriptions into formal Planning Domain Definition Languages (PDDL), and then it uses an external planner to deal with the PDDL. Finally, the generated results are transformed back into natural language by LLMs. Similarly, LLM-DP [59] utilizes LLMs to convert the observations, the current world state, and the target objectives into PDDL. Subsequently, this transformed data is passed to an external planner, which efficiently determines the final action sequence. CO-LLM [22] demonstrates that LLMs is good at generating high-level plans, but struggle with low-level control. To address this limitation, a heuristically designed external low-level planner is employed to effectively execute actions based on high-level plans.

Planning with Feedback: In many real-world scenarios, the agents need to make long-horizon planning to solve complex tasks. When facing these tasks, the above planning modules without feedback

can be less effective due to the following reasons: firstly, generating a flawless plan directly from the beginning is extremely difficult as it needs to consider various complex preconditions. As a result, simply following the initial plan often leads to failure. Moreover, the execution of the plan may be hindered by unpredictable transition dynamics, rendering the initial plan non-executable. Simultaneously, when examining how humans tackle complex tasks, we find that individuals may iteratively make and revise their plans based on external feedback. To simulate such human capability, researchers have designed many planning modules, where the agent can receive feedback after taking actions. The feedback can be obtained from environments, humans, and models, which are detailed in the following.

- *Environmental Feedback.* This feedback is obtained from the objective world or virtual environment. For instance, it could be the game's task completion signals or the observations made after the agent takes an action. In specific, ReAct [60] proposes constructing prompts using thought-act-observation triplets. The thought component aims to facilitate high-level reasoning and planning for guiding agent behaviors. The act represents a specific action taken by the agent. The observation corresponds to the outcome of the action, acquired through external feedback, such as search engine results. The next thought is influenced by the previous observations, which makes the generated plans more adaptive to the environment. Voyager [38] makes plans by incorporating three types of environment feedback including the intermediate progress of program execution, the execution error and self-verification results. These signals can help the agent to make better plans for the next action. Similar to Voyager, Ghost [16] also incorporates feedback into the reasoning and action taking processes. This

feedback encompasses the environment states as well as the success and failure information for each executed action. SayPlan [31] leverages environmental feedback derived from a scene graph simulator to validate and refine its strategic formulations. This simulator is adept at discerning the outcomes and state transitions subsequent to agent actions, facilitating SayPlan's iterative recalibration of its strategies until a viable plan is ascertained. In DEPS [33], the authors argue that solely providing information about the completion of a task is often inadequate for correcting planning errors. Therefore, they propose informing the agent about the detail reasons for task failure, allowing them to more effectively revise their plans. LLM-Planner [61] introduces a grounded re-planning algorithm that dynamically updates plans generated by LLMs when encountering object mismatches and unattainable plans during task completion. Inner Monologue [62] provides three types of feedback to the agent after it takes actions: (1) whether the task is successfully completed, (2) passive scene descriptions, and (3) active scene descriptions. The former two are generated from the environments, which makes the agent actions more reasonable.

- *Human Feedback.* In addition to obtaining feedback from the environment, directly interacting with humans is also a very intuitive strategy to enhance the agent planning capability. The human feedback is a subjective signal. It can effectively make the agent align with the human values and preferences, and also help to alleviate the hallucination problem. In Inner Monologue [62], the agent aims to perform high-level natural language instructions in a 3D visual environment. It is given the capability to actively solicit feedback from humans regarding scene descriptions. Then, the agent incorporates the human feedback into its prompts,

enabling more informed planning and reasoning. In the above cases, we can see, different types of feedback can be combined to enhance the agent planning capability. For example, Inner Monologue [62] collects both environment and human feedback to facilitate the agent plans.

- *Model Feedback.* Apart from the aforementioned environmental and human feedback, which are external signals, researchers have also investigated the utilization of internal feedback from the agents themselves. This type of feedback is usually generated based on pre-trained models. In specific, [63] proposes a self-refine mechanism. This mechanism consists of three crucial components: output, feedback, and refinement. Firstly, the agent generates an output. Then, it utilizes LLMs to provide feedback on the output and offer guidance on how to refine it. At last, the output is improved by the feedback and refinement. This output-feedback-refinement process iterates until reaching some desired conditions. SelfCheck [64] allows agents to examine and evaluate their reasoning steps generated at various stages. They can then correct any errors by comparing the outcomes. InterAct [65] uses different language models (such as ChatGPT and InstructGPT) as auxiliary roles, such as checkers and sorters, to help the main language model avoid erroneous and inefficient actions. ChatCoT [66] utilizes model feedback to improve the quality of its reasoning process. The model feedback is generated by an evaluation module that monitors the agent reasoning steps. Reflexion [12] is developed to enhance the agent’s planning capability through detailed verbal feedback. In this model, the agent first produces an action based on its memory, and then, the evaluator generates feedback by taking the agent trajectory as input. In contrast to previous studies, where the feedback is given as a scalar

value, this model leverages LLMs to provide more detailed verbal feedback, which can provide more comprehensive supports for the agent plans.

Remark. In conclusion, the implementation of planning module without feedback is relatively straightforward. However, it is primarily suitable for simple tasks that only require a small number of reasoning steps. Conversely, the strategy of planning with feedback needs more careful designs to handle the feedback. Nevertheless, it is considerably more powerful and capable of effectively addressing complex tasks that involve long-range reasoning.

2.1.4 Action Module

The action module is responsible for translating the agent’s decisions into specific outcomes. This module is located at the most downstream position and directly interacts with the environment. It is influenced by the profile, memory, and planning modules. This section introduces the action module from four perspectives: (1) Action goal: what are the intended outcomes of the actions? (2) Action production: how are the actions generated? (3) Action space: what are the available actions? (4) Action impact: what are the consequences of the actions? Among these perspectives, the first two focus on the aspects preceding the action ("before-action" aspects), the third focuses on the action itself ("in-action" aspect), and the fourth emphasizes the impact of the actions ("after-action" aspect).

Action Goal: The agent can perform actions with various objectives. Here, we present several representative examples: (1) *Task Completion.* In this scenario, the agent’s actions are aimed at accomplishing specific tasks, such as crafting an iron pickaxe in Minecraft [38] or completing a function in software development [18]. These actions usually have well-defined objectives, and each action con-

tributes to the completion of the final task. Actions aimed at this type of goal are very common in existing literature. (2) *Communication*. In this case, the actions are taken to communicate with the other agents or real humans for sharing information or collaboration. For example, the agents in ChatDev [18] may communicate with each other to collectively accomplish software development tasks. In Inner Monologue [62], the agent actively engages in communication with humans and adjusts its action strategies based on human feedback. (3) *Environment Exploration*. In this example, the agent aims to explore unfamiliar environments to expand its perception and strike a balance between exploring and exploiting. For instance, the agent in Voyager [38] may explore unknown skills in their task completion process and continually refine the skill execution code based on environment feedback through trial and error.

Action Production: Different from ordinary LLMs, where the model input and output are directly associated, the agent may take actions via different strategies and sources. In the following, we introduce two types of commonly used action production strategies. (1) *Action via Memory Recollection*. In this strategy, the action is generated by extracting information from the agent memory according to the current task. The task and the extracted memories are used as prompts to trigger the agent actions. For example, in Generative Agents [20], the agent maintains a memory stream, and before taking each action, it retrieves recent, relevant and important information from the memory stream to guide the agent actions. In GITM [16], in order to achieve a low-level sub-goal, the agent queries its memory to determine if there are any successful experiences related to the task. If similar tasks have been completed previously, the agent invokes the previously

successful actions to handle the current task directly. In collaborative agents such as ChatDev [18] and MetaGPT [23], different agents may communicate with each other. In this process, the conversation history in a dialog is remembered in the agent memories. Each utterance generated by the agent is influenced by its memory. (2) *Action via Plan Following*. In this strategy, the agent takes actions following its pre-generated plans. For instance, in DEPS [33], for a given task, the agent first makes action plans. If there are no signals indicating plan failure, the agent will strictly adhere to these plans. In GITM [16], the agent makes high-level plans by decomposing the task into many sub-goals. Based on these plans, the agent takes actions to solve each sub-goal sequentially to complete the final task.

Action Space: Action space refers to the set of possible actions that can be performed by the agent. In general, we can roughly divide these actions into two classes: (1) external tools and (2) internal knowledge of the LLMs. In the following, we introduce these actions more in detail.

- *External Tools*. While LLMs have been demonstrated to be effective in accomplishing a large amount of tasks, they may not work well for the domains which need comprehensive expert knowledge. In addition, LLMs may also encounter hallucination problems, which are hard to be resolved by themselves. To alleviate the above problems, the agents are empowered with the capability to call external tools for executing action. In the following, we present several representative tools which have been exploited in the literature.

(1) *APIs*. Leveraging external APIs to complement and expand action space is a popular paradigm in recent years. For example, HuggingGPT [13] integrates HuggingFace’s vast model ecosystem to tackle complex user tasks. Similarly, WebGPT [67]

proposes to automatically generate queries to extract relevant content from external web pages when responding to user request. TPTU [68] explores the potential of LLMs to address intricate tasks through strategic task planning and API-based tools. Gorilla [69] introduces a fine-tuned LLM capable of generating precise input arguments for API calls, effectively mitigating hallucination issues during external API usage. ToolFormer [15] employs self-supervised learning to determine when and how to invoke external tools, using demonstrations of tool APIs for training. API-Bank [70] offers a comprehensive benchmark with a diverse collection of API tools to systematically evaluate tool-augmented LLMs, alongside robust training datasets designed to enhance their integration capabilities. ToolLLaMA [14] proposes a tool-use framework encompassing data collection, training, and evaluation, with the resulting fine-tuned model excelling across a wide array of APIs. RestGPT [71] connects LLMs with RESTful APIs, which follow widely accepted standards for web services development, making the resulting program more compatible with real-world applications. TaskMatrix.AI [72] connects LLMs with an extensive ecosystem of APIs to support task execution. At its core lies a multimodal conversational foundational model that interacts with users, understands their goals and context, and then produces executable code for particular tasks. In essence, these intelligent agents strategically harness external APIs as versatile tools, systematically expanding their action space and transcending the inherent limitations of traditional language models by integrating diverse computational capabilities.

(2) *Databases & Knowledge Bases.* Integrating external database or knowledge base enables agents to obtain specific domain information for generating more realistic actions. For example, ChatDB [40]

employs SQL statements to query databases, facilitating actions by the agents in a logical manner. MRKL [73] and OpenAGI [74] incorporate various expert systems such as knowledge bases and planners to access domain-specific information.

(3) *External Models.* Previous studies often utilize external models to expand the range of possible actions. In comparison to APIs, external models typically handle more complex tasks. Each external model may correspond to multiple APIs. For example, ViperGPT [75] firstly uses Codex, which is implemented based on language model, to generate Python code from text descriptions, and then executes the code to complete the given tasks. ChemCrow [76] is an LLM-based chemical agent designed to perform tasks in organic synthesis, drug discovery, and material design. It utilizes seventeen expert-designed models to assist its operations. MM-REACT [77] integrates various external models, such as VideoBERT for video summarization, X-decoder for image generation, and SpeechBERT for audio processing, enhancing its capability in diverse multimodal scenarios.

• *Internal Knowledge.* In addition to utilizing external tools, many agents rely solely on the internal knowledge of LLMs to guide their actions. We now present several crucial capabilities of LLMs that can support the agent to behave reasonably and effectively. (1) *Planning Capability.* Previous work has demonstrated that LLMs can be used as decent planners to decompose complex tasks into simpler ones [45]. Such a capability of LLMs can be even triggered without incorporating examples in the prompts [46]. Building on the planning capability of LLMs, DEPS [33] develops a Minecraft agent, which can solve complex tasks via sub-goal decomposition. Similar agents like GITM [16] and Voyager [38] also heavily rely on the planning ca-